



Politechnika Wrocławska

Sterowanie procesami dyskretnymi Automatyka i Robotyka, W12N	Zadanie nr 3 Problem maszyn równoległych	
Jakub Borsukiewicz 278319 Jan Farbotko 278325	Termin zajęć: Wt. Tn. 11:15	Prowadzący: dr inż. Agnieszka Wielgus

Spis treści

1	Opis problemu matematycznego	3
2	Zaimplementowane algorytmy	3
2.1	Algorytm LSA	3
2.2	Algorytm LPT	3
2.3	Programowanie dynamiczne	4
2.4	Przegląd zupełny	4
2.5	Algorytm PTAS	5
2.6	Algorytm FPTAS	5
3	Testy eksperymentalne	6
3.1	Wyniki testów działania algorytmów dla dwóch maszyn	6
3.2	Wyniki testów działania algorytmów dla trzech maszyn	8
4	Wnioski	9

1 Opis problemu matematycznego

Dany jest zbiór n niepodzielnych zadań $J = 1, \dots, j, \dots, n$ i m równoległych, identycznych maszyn $M = 1, \dots, i, \dots, m$, gdzie każde zadanie j charakteryzuje się znanym czasem wykonania p_j .

Zakłada się, że każde zadanie może być wykonywane dokładnie przez jedną maszynę w danym momencie oraz zadania nie mogą być przerywane.

Celem problemu jest znalezienie takiego przyporządkowania zadań do maszyn, aby zminimalizować czas zakończenia ostatniego zadania (te które wykonają się jako ostatnie). Kryterium to długość uszeregowania oznaczona jako C_{max} , gdzie:

$$C_{max} = \max_{j \in J} C_j$$

gdzie C_j jest czasem zakończenia wykonania zadania j .

2 Zaimplementowane algorytmy

Zaimplementowano różne algorytmy, których celem była minimalizacja kryterium długości uszeregowania zadań na maszynach.

2.1 Algorytm LSA

- **Zasada działania:** Algorytm przetwarza zadania w kolejności ich występowania z góry ustalonej listy. W każdym kroku pobierane jest kolejne zadanie i przypisywane do tej maszyny, która w danym momencie jest najmniej obciążona.
- **Cechy szczególne:** Jest to najprostsza heurystyka zachłanna działająca w trybie online. Gwarantuje współczynnik aproksymacji na poziomie $2 - \frac{1}{m}$, co oznacza, że wyznaczone uszeregowanie będzie co najwyżej dwukrotnie gorsze od optymalnego.
- **Złożoność obliczeniowa:** $O(n \cdot m)$, wynika z konieczności pełnego, liniowego przeszukania wektora m maszyn dla każdego z n zadań w celu znalezienia wartości minimalnej (możliwe jest osiągnięcie złożoności $O(n \log m)$, przy zastosowaniu kolejki priorytetowej. Jednak przy użyciu 2-3 maszyn nie ma to dużego znaczenia.).

2.2 Algorytm LPT

- **Zasada działania:** Jest to modyfikacja algorytmu listowego (LSA). W pierwszej fazie wszystkie zadania są sortowane malejąco według ich

czasów wykonywania p_j . Następnie, w fazie konstrukcyjnej, zadania są sekwencyjnie przydzielane do maszyn o aktualnie najmniejszym obciążeniu.

- **Cechy szczególne:** Dzięki uszeregowaniu najdłuższych zadań na samym początku, algorytm unika sytuacji, w której duże zadanie na końcu drastycznie wydłuża $C_{\max}$. Zapewnia doskonały współczynnik aproksymacji równy $\frac{4}{3} - \frac{1}{3m}$.
- **Złożoność obliczeniowa:** $O(n \log n + n \cdot m)$, gdzie człon $n \log n$ wynika z konieczności wstępnego posortowania zadań.

2.3 Programowanie dynamiczne

- **Zasada działania:** Algorytm dekomponuje problem na podproblemy, wyznaczając stany reprezentujące możliwe wektory obciążeń maszyn po przydzieleniu podzbioru zadań. Dla stałej liczby maszyn m , algorytm sprawdza wszystkie dopuszczalne konfiguracje podziału, korzystając z zapamiętanych wyników wcześniejszych kroków rekurencyjnych.
- **Cechy szczególne:** Pozwala na znalezienie optymalnego rozwiązania problemu. Ze względu na pseudowielomianowy czas działania, jest skuteczny tylko dla instancji o niewielkich czasach zadań p_j lub małej liczbie maszyn.
- **Złożoność obliczeniowa:** $O(n \cdot (\sum p_j)^{m-1})$, co oznacza, że jest ona wykładnicza względem liczby maszyn m , ale wielomianowa względem sumy czasów zadań dla stałego m .

2.4 Przegląd zupełny

- **Zasada działania:** Algorytm generuje wszystkie możliwe przyporządkowania n zadań do m maszyn. Dla każdego wygenerowanego wariantu (podziału) obliczany jest czas pracy każdej maszyny, a następnie wyznaczana jest wartość kryterium $C_{\max}$.
- **Cechy szczególne:** Gwarantuje znalezienie rozwiązania optymalnego poprzez sprawdzenie całego drzewa rozwiązań. Nie wymaga sortowania ani zaawansowanych struktur danych.
- **Złożoność obliczeniowa:** $O(m^n \cdot n)$ - czas obliczeń rośnie wykładniczo wraz z liczbą zadań, co ogranicza praktyczne zastosowanie algorytmu do bardzo małych instancji testowych.

2.5 Algorytm PTAS

- **Zasada działania:** Algorytm na początku sortuje wszystkie zadania malejąco według czasów ich wykonania. Następnie wybiera k największych zadań i przypisuje je do maszyn w sposób optymalny, stosując algorytm przeglądu zupełnego. Pozostałe zadania w sposób zachłanny są dokładane do maszyn, przy użyciu algorytmu listowego (LSA).
- **Cechy szczególne:** Pełna kontrola nad kompromisem pomiędzy czasem obliczeń a dokładnością. Zwiększanie wartości parametru k pozwala na zmniejszenie błędu względnego ϵ (szacowanego jako ok. $1/k$), ale odbywa się to kosztem drastycznego wydłużenia czasu pracy algorytmu.
- **Złożoność obliczeniowa:** $O(n \log n + m^k)$ – gdzie człon $n \log n$ odpowiada za wstępne sortowanie, natomiast człon wykładniczy m^k wynika z konieczności przejrzenia wszystkich kombinacji przydziału dla k największych zadań na m maszynach.

2.6 Algorytm FPTAS

- **Zasada działania:** Algorytm opiera się na technice zmniejszania dokładności danych wejściowych w celu uproszczenia obliczeń. Wszystkie oryginalne czasy zadań p_j są dzielone przez stały współczynnik skalujący k i zaokrąglane w dół do liczb całkowitych. Dla tak uproszczonej instancji uruchamiany jest algorytm programowania dynamicznego. Otrzymany w ten sposób optymalny podział zadań przenosi się bezpośrednio na oryginalne dane.
- **Cechy szczególne:** Reprezentuje najwyższą i najsilniejszą klasę algorytmów przybliżonych. Gwarantuje, że błąd końcowego rozwiązania nie przekroczy wartości ϵ , a jednocześnie czas działania programu pozostanie bezpieczny (wielomianowy) zarówno względem rozmiaru danych n , jak i odwrotności błędu $1/\epsilon$.
- **Złożoność obliczeniowa:** Wielomianowa, zależna bezpośrednio od rozmiaru problemu oraz parametru błędu, najczęściej przyjmująca postać rzędu $O(n^3/\epsilon^2)$ lub zbliżoną.

3 Testy eksperymentalne

Przeprowadzono testy na algorytmach w celu analizy porównawczej ich wydajności czasowej oraz jakości generowanych rozwiązań przez algorytmy w oparciu o wyznaczone wartości kryterium C_{max} . Testy wykonano na procesorze intel core i7-5600U, oraz 8GB pamięci RAM.

3.1 Wyniki testów działania algorytmów dla dwóch maszyn

Przeprowadzono testy dla różnych algorytmów na różnych wielkościach instancji oraz o różnych zakresach długości zadań dla dwóch maszyn.

Wyniki przedstawiają tabele 1 i 2:

Tabela 1: Wyniki eksperymentów dla LSA, LPT, PD i PZ

Rozmiar Instancji dla 2 maszyn (liczba Zadań) Przedział p_j	Wartość Kryterium LSA	Czas Działania Alg. LSA [ms]	Wartość Kryterium LPT	Czas Działania Alg. LPT [ms]	Wartość Kryterium PD i PZ	Czas Działania Alg. PD [ms]	Czas Działania Alg. PZ [ms]
8 [1-10]	21 [0.00%]	0.009389	21 [0.00%]	0.008325	21	0.035425	0.117561
8 [10-20]	62 [0.00%]	0.029008	62 [0.00%]	0.026275	62	0.143307	0.17071
10 [1-10]	28 [0.00%]	0.016406	28 [0.00%]	0.019131	28	0.093525	0.691893
10 [10-20]	79 [1.28%]	0.023251	78 [0.00%]	0.029797	78	0.106793	0.702812
12 [1-10]	34 [6.25%]	0.018763	32 [0.00%]	0.015611	32	0.08742	2.2482
12 [10-20]	97 [6.59%]	0.029246	91 [0.00%]	0.018767	91	0.150896	2.469
20 [1-10]	55 [0.00%]	0.063413	55 [0.00%]	0.052335	55	0.419831	578.144
20 [10-20]	157 [1.29%]	0.026637	155 [0.00%]	0.018884	155	0.333809	560.353
20 [20-50]	360 [2.56%]	0.016771	351 [0.00%]	0.015957	351	0.739773	559.59
20 [50-100]	766 [2.82%]	0.023404	745 [0.00%]	0.016615	745	2.05192	560.867
30 [1-10]	84 [1.20%]	0.025384	83 [0.00%]	0.022329	83	0.398182	—
30 [10-20]	237 [1.72%]	0.025002	233 [0.00%]	0.028857	233	0.98842	—
30 [20-50]	550 [3.38%]	0.019684	532 [0.00%]	0.016767	532	2.10357	—
30 [50-100]	1120 [0.27%]	0.025507	1118 [0.09%]	0.024576	1117	3.43136	—
20 [1-1000]	3889 [0.03%]	0.022154	3889 [0.03%]	0.017503	3888	10.3054	—

Tabela 2: Wyniki eksperymentów dla PTAS i FPTAS

Rozmiar Instancji dla 2 maszyn (liczba Zadań) Przedział p_j	Wartość Kryterium PTAS $k=n/2$	Czas Działania PTAS $k=n/2$	Wartość Kryterium PTAS $K=2n/3$	Czas Działania PTAS $K=2n/3$	Wartość Kryterium PTAS $K=3n/4$	Czas Działania PTAS $K=3n/4$	Wartość Kryterium FPTAS $K=2$	Czas Działania FPTAS $K=2$	Wartość Kryterium FPTAS $K=3$	Czas Działania FPTAS $K=4$	Wartość Kryterium FPTAS $K=4$	Czas Działania FPTAS $K=4$
8 [1-10]	21 [0.00%]	0.019089	21 [0.00%]	0.026541	21 [0.00%]	0.033738	22 [4.76%]	0.023355	22 [4.76%]	0.022749	22 [4.76%]	0.02004
8 [10-20]	62 [0.00%]	0.034551	62 [0.00%]	0.040929	62 [0.00%]	0.058512	62 [0.00%]	0.083271	64 [3.23%]	0.609988	64 [3.23%]	0.031799
10 [1-10]	28 [0.00%]	0.081545	28 [0.00%]	0.145528	28 [0.00%]	0.184179	28 [0.00%]	0.090191	28 [0.00%]	0.140807	34 [21.43%]	0.072882
10 [10-20]	78 [0.00%]	0.039888	78 [0.00%]	0.048444	78 [0.00%]	0.06937	78 [0.00%]	0.066062	79 [1.28%]	0.044203	79 [1.28%]	0.036637
12 [1-10]	32 [0.00%]	0.104122	32 [0.00%]	0.200254	32 [0.00%]	0.316851	32 [0.00%]	0.058813	33 [3.13%]	0.033304	34 [6.25%]	0.025082
12 [10-20]	91 [0.00%]	0.069105	91 [0.00%]	0.159738	91 [0.00%]	0.257242	91 [0.00%]	0.122898	94 [3.30%]	0.124302	94 [3.30%]	0.064922
20 [1-10]	55 [0.00%]	0.93045	55 [0.00%]	7.64472	55 [0.00%]	22.7755	56 [1.82%]	0.111027	58 [5.45%]	0.067884	61 [10.91%]	0.05303
20 [10-20]	155 [0.00%]	1.38837	155 [0.00%]	6.556	155 [0.00%]	22.7337	155 [0.00%]	0.185986	155 [0.00%]	0.161637	155 [0.00%]	0.123612
20 [20-50]	351 [0.00%]	1.39114	351 [0.00%]	6.23069	351 [0.00%]	23.8224	352 [0.28%]	0.436813	352 [0.28%]	0.26999	353 [0.57%]	0.195259
20 [50-100]	745 [0.00%]	1.1452	745 [0.00%]	6.4665	745 [0.00%]	29.8715	746 [0.13%]	0.772204	746 [0.13%]	0.519435	746 [0.13%]	0.411153
30 [1-10]	83 [0.00%]	42.0676	83 [0.00%]	998.991	83 [0.00%]	3588.11	83 [0.00%]	0.189032	86 [3.61%]	0.113902	94 [13.25%]	0.083192
30 [10-20]	233 [0.00%]	43.1463	233 [0.00%]	1009.69	233 [0.00%]	3635.09	234 [0.43%]	0.412631	233 [0.00%]	0.349725	234 [0.43%]	0.332805
30 [20-50]	532 [0.00%]	43.0659	532 [0.00%]	1022.51	532 [0.00%]	3626.85	532 [0.00%]	0.969233	533 [0.19%]	0.710396	533 [0.19%]	0.41881
30 [50-100]	1117 [0.00%]	43.0467	1117 [0.00%]	1057.56	1117 [0.00%]	3653.69	1118 [0.09%]	2.2967	1117 [0.00%]	1.70541	1125 [0.72%]	1.15268
20 [1-1000]	3888 [0.00%]	1.1126	3888 [0.00%]	5.98334	3888 [0.00%]	24.154	3888 [0.00%]	4.0449	3891 [0.08%]	2.46829	3891 [0.08%]	1.87416

Na podstawie wyników można zaobserwować:

- Zarówno algorytm LSA, jak i LPT charakteryzują się niezwykle krótkim czasem działania (poniżej 0.1 ms dla wszystkich badanych instancji). Zgodnie z oczekiwaniami, wstępne posortowanie zadań w regule LPT drastycznie poprawia jakość rozwiązań – w większości prób błąd względny LPT wynosił dokładnie 0.00%, podczas gdy LSA regularnie generował błędy do aż nawet 6.59% (dla instancji 12 [10-20]).
- Algorytm PZ, mimo gwarancji optymalności jest skrajnie niewydajny czasowo. Dla instancji o rozmiarze $n = 20$ czas obliczeń gwałtownie wzrósł do ponad 560 ms, natomiast dla instancji o rozmiarze $n = 30$ nie wpisano wyniku ze względu na zbyt długi czas obliczeń. Potwierdza to teoretyczną wykładniczą złożoność algorytmu o postaci $O(m^n \cdot n)$.
- Algorytm PD zawsze odnajduje rozwiązanie optymalne, a jego czas działania zależy nie tylko od liczby zadań n , ale przede wszystkim od sumy ich czasów wykonania ($\sum p_j$). Dla małych zakresów wartości (np. interval [1-10]) czas działania jest bardzo krótki (np. 0.398 ms dla $n = 30$). Jednakże dla instancji 20 [1-1000], gdzie występują duże wartości, czas wzrósł aż do 10.305 ms, co idealnie obrazuje pseudowiełomianowość algorytmu.
- W algorytmie PTAS wraz ze wzrostem parametru K błąd aproksymacji maleje do zera, lecz kosztem drastycznego (wykładniczego) wzrostu

czasu obliczeń. Dla $n = 30$ i $K = n/2$ czas wynosi ok. 43 ms, przy $K = 2n/3$ rośnie do ok. 1000 ms, a przy $K = 3n/4$ osiąga gigantyczną wartość ponad 3500 ms (ponad 3.5 sekundy) przy dokładnie tej samej (zerowej) wartości błędu. W przeprowadzonych testach nie można stwierdzić, że zwiększanie parametru K polepsza działanie algorytmu. Wynika to z wybranych instancji (są zbyt małe) lub z powodu zbyt dużej wartości parametru.

- Algorytm FPTAS wykazuje doskonałe zachowanie pod kątem stabilności czasowej. Nawet przy najbardziej rygorystycznym kryterium błędu ($K = 4$, co odpowiada najmniejszemu dopuszczalnemu błędowi ϵ), maksymalny czas obliczeń dla $n = 30$ nie przekroczył 2 ms. W porównaniu z PTAS, FPTAS dla dużych instancji działa setki, a nawet tysiące razy szybciej, utrzymując błąd na poziomie ułamka procenta (maksymalnie 13.25% dla skrajnie niekorzystnej instancji 30 [1-10], w większości przypadków poniżej 1%).

3.2 Wyniki testów działania algorytmów dla trzech maszyn

Przeprowadzono testy dla różnych algorytmów na różnych wielkościach instancji oraz o różnych zakresach długości zadań dla trzech maszyn.

Wyniki przedstawia tabela 3

Tabela 3: Wyniki eksperymentów dla PTAS, PD i PZ

Rozmiar Instancji dla 3 maszyn (liczba Zadań) Przedział p_j	Wartość Kryterium PTAS $k=n/2$	Czas Działania PTAS $k=n/2$	Wartość Kryterium PTAS $K=2n/3$	Czas Działania PTAS $K=2n/3$	Wartość Kryterium PTAS $K=3n/4$	Czas Działania PTAS $K=3n/4$	Wartość Kryterium PD i PZ	Czas Działania Alg. PD [ms]	Czas Działania Alg. PZ [ms]
8 [1-10]	14 [0.00%]	0.061747	14 [0.00%]	0.132089	14 [0.00%]	0.354534	14	2.40665	3.66279
8 [10-20]	43 [0.00%]	0.07523	43 [0.00%]	0.209383	43 [0.00%]	0.494069	43	10.8743	3.87188
10 [1-10]	19 [0.00%]	0.349833	19 [0.00%]	1.02937	19 [0.00%]	1.14082	19	4.77894	29.6874
10 [10-20]	53 [1.92%]	0.26358	53 [1.92%]	0.601436	52 [0.00%]	1.65673	52	22.2496	32.8558
12 [1-10]	21 [0.00%]	0.667747	21 [0.00%]	5.15882	21 [0.00%]	12.3164	21	5.95603	276.258
12 [10-20]	61 [0.00%]	0.753487	61 [0.00%]	6.00563	61 [0.00%]	15.6018	61	34.833	287.874
20 [1-10]	37 [0.00%]	64.8621	37 [0.00%]	1365.28	37 [0.00%]	10512.8	37	25.0341	1910100
20 [10-20]	104 [0.00%]	67.6053	104 [0.00%]	1373.77	104 [0.00%]	10506.9	104	143.659	1903540
20 [20-50]	234 [0.00%]	66.0965	234 [0.00%]	1353.45	234 [0.00%]	10465	234	671.057	1904390
20 [50-100]	497 [0.00%]	64.7646	497 [0.00%]	1357.68	497 [0.00%]	10443.2	497	2790.03	1917310

Na podstawie wyników można zaobserwować:

- W algorytmie PTAS zwiększanie rozmiaru podproblemu rozwiązywanego dokładnie pozwala na stopniowe eliminowanie błędu aproksymacji. Efekt ten widać na trudnej instancji 10 [10-20]: dla $k = n/2$ oraz $K = 2n/3$ algorytm generuje błąd względny na poziomie 1.92%, natomiast zwiększenie parametru do $K = 3n/4$ pozwala na odnalezienie rozwiązania optymalnego. Dla pozostałych małych instancji relatywnie duże wartości parametru k pozwalają od razu osiągnąć pełną optymalność. Niemniej jednak zwiększenie parametru k powoduje zwiększenie się czasu potrzebnego na obliczenia, a szczególnie w przypadku dużych instancji. Dla 20 zadań ten wzrost jest widoczny - z ok. 65 ms dla $K = n/2$ do nawet ponad 10000 ms dla $K = 3n/4$.
- Algorytm Przeglądu Zupełnego wykazuje skrajną wrażliwość na zwiększenie liczby maszyn. Podczas gdy dla małych instancji ($n = 8, 10$) czasy działania oscylują w granicach kilku lub kilkunastu milisekund, dla instancji o rozmiarze $n = 20$ czas obliczeń gwałtownie skacze do poziomu około 1 910 000 ms (czyli ponad 31 minut). Jest to idealna ilustracja teoretycznej złożoności wykładniczej $O(m^n \cdot n)$, gdzie zmiana podstawy potęgi z 2 na 3 drastycznie ogranicza stosowalność tej metody.
- Algorytm PD, podobnie jak PZ, zawsze wyznacza rozwiązanie optymalne, jednak jego czas działania zależy bezpośrednio od sumy czasów zadań ($\sum p_j$). Dla stałej liczby zadań $n = 20$, czas obliczeń wzrasta od 25.03 ms dla przedziału [1-10] do aż 2790.03 ms dla przedziału [50-100]. Wynika to z faktu, że dla 3 maszyn złożoność PD wynosi $O(n \cdot (\sum p_j)^2)$ – podniesienie sumy czasów do kwadratu potęguje narzut obliczeniowy przy przejściu do wyższych przedziałów wartości.

4 Wnioski

- Przeprowadzone badania empiryczne jednoznacznie potwierdzają teoretyczną klasyfikację problemu szeregowania zadań na maszynach identycznych jako problemu NP-trudnego. Gwałtowny, wykładniczy przyrost czasu działania algorytmów dokładnych (PZ oraz PD) wraz ze wzrostem parametrów instancji (n lub $\sum p_j$) udowadnia, że metody te są całkowicie nieprzydatne w rzeczywistych systemach sterowania procesami dyskretnymi o dużej skali.

- Eksperymenty pokazały, że heurystyka LPT stanowi najlepszy wybór, jeżeli kluczowym czynnikiem jest natychmiastowy czas odpowiedzi (np. systemy czasu rzeczywistego). Oferuje ona wyniki optymalne lub bliskie optymalnym w czasie mierzonym w ułamkach milisekund.
- W kontekście ograniczeń algorytmów dokładnych, kluczowego znaczenia nabierają wielomianowe schematy aproksymacyjne (PTAS oraz FPTAS). Pozwalają one na zachowanie pełnej kontroli nad kompromisem pomiędzy oczekiwaną dokładnością rozwiązania (ϵ) a kosztem czasowym procesora.
- Zgodnie z teorią, FPTAS eliminuje wadę schematu PTAS, jaką jest wykładnicza zależność czasu działania od odwrotności oczekiwanego błędu ($1/\epsilon$). Skalowanie danych wejściowych i uruchomienie programowania dynamicznego na liczbach o mniejszej precyzji pozwala na elastyczne sterowanie procesem obliczeniowym, czyniąc FPTAS potężnym i skalowalnym narzędziem optymalizacyjnym.
- Przejście z $m = 2$ na $m = 3$ zamienia podstawę potęgi w przeglądzie zupełnym z 2^n na 3^n , co doprowadziło do paraliżu algorytmu PZ dla $n = 20$ i wydłużenia czasu pracy do ponad 31 minut. W przypadku programowania dynamicznego, wzrost parametru m podnosi wykładnik potęgowy przestrzeni stanów algorytmu ($O(n \cdot (\sum p_j)^{m-1})$), sprawiając, że PD staje się skrajnie wrażliwe na zakres generowanych czasów zadań (wzrost z 143.65 ms do 2790.03 ms dla instancji $n = 20$ przy zmianie interwału na [50-100]).
- Eksperyment dla $m = 3$ dla algorytmu PTAS wykazał wadę praktyczną tego schematu, która nie była widoczna jak przy dwóch maszynach. Ze względu na fakt, że faza optymalnego rozmieszczenia k największych zadań bazuje na przeglądzie zupełnym o złożoności $O(m^k)$, uzależnienie parametru k relatywnie od rozmiaru instancji (np. $K = 3n/4$) powoduje, że przy trzech maszynach algorytm staje się nieużyteczny inżyniersko z powodu zbyt dużego czasu obliczeń. W rzeczywistych implementacjach PTAS parametr k musi być wartością stałą.